

SOP – Foci Detection Workflow

Inhalt

General	2
Mask Creation (Cellpose_DAPI_Mask_Creation.ipynb)	2
Preparation.....	2
Execution	3
Mask Cleanup (Cellpose_DAPI_Mask_Cleanup.ipynb)	3
Preparation 1.....	3
Execution 1	4
Preparation 2.....	4
Execution 2	4
Foci Detection (Foci Detection --> Foci_Detection_main.ipynb)	5
Configuration (Foci Detection --> config.yaml)	5
Parameter space generation	16
Foci area detection threshold	19
Picture Analysis	20

General

This SOP is explaining how to run the foci detection workflow step by step.

Note: All the recommendations in this SOP are not based on any scientific testing.

Mask Creation (Cellpose_DAPI_Mask_Creation.ipynb)

In the first step, the masks for the nuclei are created using Cellpose. With the results from this and the next step, the program will be able to identify and quantify individual nuclei for downstream analysis.

Preparation

1. Model Path

The Cellpose model needs to be saved locally on the computer (not on GitHub/UZH server). Add the model path to the user inputs at the top of the third cell.

Example:

```
# Using Cellpose on the previously defined pictures

# Load all DAPI images into memory as numpy arrays
# io.imread() handles various formats (tif, png, etc.) and returns numpy arrays
# NOTE: Variable shadowing - 'DAPI_dir_path' in comprehension shadows the outer variable
# Consider renaming to 'img_path' for clarity:
# imgs = [io.imread(str(img_path)) for img_path in DAPI_list]
imgs = [io.imread(str(DAPI_dir_path)) for DAPI_dir_path in DAPI_list]

# Load your custom-trained DAPI segmentation model
# This model was trained earlier using the training notebook
model = models.CellposeModel(
    gpu=True,
    pretrained_model=r'C:\Users\v.aubry\Desktop\Microscopy_Data\Jupyter_Notebook\models\DAPI_model'
)
```

Note: The path needs to have the format `r'Path'` to handle backslashes correctly.

2. Data Path

The data (pictures) can be stored either on the UZH server or the local computer. Add the data directory path to the top of the second cell.

Example:

```
#Data#

# Path to the microscopy dataset on network drive
# This appears to be a U2OS cell experiment with multiple stains:
# gH2AX (488nm), 53BP1 (568nm), EdU (647nm) - we're using DAPI channel for nuclear segmentation
DAPI_dir = r'Y:\Group Members\Valentin Aubry\01_Data\Final_Test'
```

Note: The path needs to have the format `r'Path'` to handle backslashes correctly.

3. File Naming Convention

Ensure that DAPI images follow the naming convention with "DAPI SEM.tif" as the ending to all the images from the DAPI channel. The program will automatically detect and process all matching files in the specified directory.

Execution

Run all cells in the Jupyter notebook sequentially from top to bottom.

The resulting masks are automatically saved in the same directory as the input images with corresponding filenames (e.g., image_DAPI SEM.tif → image_DAPI SEM_seg.npy).

Runtime:

For ~700 images, expect a runtime of approximately 3 hours (roughly 14 seconds per image).

Progress is displayed in the output of the final cell during execution.

Mask Cleanup (Cellpose_DAPI_Mask_Cleanup.ipynb)

In the second step, the masks created by Cellpose are refined to remove incomplete nuclei and artifacts. This ensures only complete, properly-sized nuclei are included in the downstream analysis.

Preparation 1

1. Data Path

Specify the directory containing the Cellpose mask files (_seg.npy) generated in the previous step at the top of the second cell. This is typically the same folder as the original images.

Example:

```
# Data Loading --> Choosing a folder

# Path to the microscopy dataset on network drive
# The 'r' prefix creates a raw string, preventing backslashes from being interpreted
# as escape characters (e.g., \n would be newline without 'r')
folder_path = r'Y:\Group Members\Valentin Aubry\01_Data\Final_Test'
```

Note: The path must use the raw string format `r'Path'` to handle backslashes correctly.

2. Zoom Value

Set the maximum value for the x-axis in the zoomed picture in the 5th cell. This only exists to get a better idea visually where to set the threshold for filtering out small fragments in the next section.

Example:

```
# ===== Plot 3: Zoomed view to see small nuclei distribution =====
# Zooming in helps identify where to set the minimum size threshold
zoom_max = 2500 # Can be changed, based on the data
for i, areas in enumerate(all_areas_noborder):
    axes[2].scatter(areas, [i+1]*len(areas), alpha=0.6, s=20, color='orange')
axes[2].set_xlabel('DAPI Area (pixels)', fontsize=12)
axes[2].set_ylabel('Picture Index', fontsize=12)
axes[2].set_title('After Border Removal (Zoomed)', fontsize=14, fontweight='bold')
axes[2].set_xlim(x_min, zoom_max)
axes[2].set_ylim(0, y_max)
```

Note: If you want to change the zoom value after running all cells, update it in the user input cell, then re-run that cell and the visualization cell (the cell just before the next user input section).

Execution 1

Run the first 5 cells.

Border-touching nuclei are removed and size distribution plots are generated to help determine the appropriate threshold.

Runtime:

For ~700 images a runtime of around 20 minutes is expected (roughly 1.5 seconds per image (mask))

Preparation 2

1. Size Threshold

Set an appropriate size threshold to filter out debris and artifacts in the 6th cell.

Example:

```
# Cellpose DAPI Mask Cleanup - Step 2: Size Threshold
# Remove small fragments that are likely debris, segmentation artifacts, or apoptotic bodies
# rather than real nuclei

start_time = time.time()

# Reset tracking lists for this step
all_areas_noborder = [] # Areas from border-cleaned masks (input to this step)
all_areas_clean = []    # Areas after size filtering (output of this step)

# Minimum nucleus size threshold in pixels
# Adjust based on the distribution visualized in Cell 4
# Objects smaller than this are considered debris/artifacts
threshold = 500 # <-- TUNE THIS VALUE based on your data
```

Note: The version of the mask that still has all the fragments is saved separately, so if you want to filter it differently later, you can just continue from here (+ adding the data path at the start and running that cell).

Execution 2

Run the remaining cells.

The cleaned masks are saved with the original filename, overwriting the initial Cellpose output (e.g., image_DAPI_SEM_seg.npy is updated with the cleaned version).

Runtime:

For ~700 images, expect a runtime of approximately 5 minutes:

Progress and nuclei counts are displayed in the cell outputs during execution.

Output:

The final cleaned masks contain only nuclei and are saved into the same directory like the images.

Example of the saved masks to the picture:

 ATR3_8h--W00053--P00001--Z00000--T00000--DAPI SEM.tif	Original Picture
 ATR3_8h--W00053--P00001--Z00000--T00000--DAPI SEM_seg.npy	Final Mask
 ATR3_8h--W00053--P00001--Z00000--T00000--DAPI SEM_seg_noborder.npy	Intermediate Mask

Note: Two versions of masks will be present in the directory:

*_seg.npy - Final cleaned masks (border-removed + size-filtered)

*_seg_noborder.npy - Intermediate masks (border-removed only)

The _noborder.npy files allow you to easily re-run size threshold filtering with different values without starting from the beginning. Otherwise this file is not used.

[Foci Detection \(Foci Detection --> Foci_Detection_main.ipynb\)](#)

Configuration ([Foci Detection --> config.yaml](#))

Before running the main program, open the config.yaml file and configure the following sections:

DATA INPUT

folder_path

```
data:
  # Path to folder containing microscopy images
  # Required files per image position:
  #   *_TRITC_SEM.tif, *_FITC_SEM.tif, *_Cy5_SEM.tif
  #   *_DAPI_SEM.tif, *_DAPI_SEM_seg.npy
  folder_path: "Y:\\Group Members\\Valentin Aubry\\01_Data\\Final_Test"
```

Specify the directory containing the Microscopy images (File endings: *_TRITC_SEM.tif, *_FITC_SEM.tif, *_Cy5_SEM.tif, *_DAPI_SEM.tif) and the Cleaned masks from the previous step (*_DAPI_SEM_seg.npy).

This will probably be the same path that was used in the mask creation and mask cleanup steps.

Note: Use double backslashes (\\) in YAML format (different from before).

PARAMETER SPACE

generate_new

```
parameter_space:
  # Generate new parameter spaces via interactive annotation?
  # true:  Manually annotate nuclei, generate new spaces
  # false: Load existing spaces from paths below
  generate_new: false
```

This setting controls if a new parameter iteration space is created before the foci detection. This parameter iteration space is used to create a personalized set of parameters for each dataset to increase the accuracy of the foci detection.

Set this to true if:

- You have a new data set.
- The detection result was unexpected and you want to re-run the foci detection with a different set of parameters (caution: it will override the old parameter space in the same folder).

Set this to false if:

- You are continuing to analyze a data set for which a parameter iteration space already exists.

base_path, tritc_path, fitc_path

```
# Base directory where NEW parameter spaces will be saved
# Only used when generate_new: true
base_path: "Y:\\Group Members\\Valentin Aubry\\01_Data\\Final_Test\\Parameters\\Complete_KDE"

# Paths to existing parameter space directories
# Only used when generate_new: false
tritc_path: "Y:\\Group Members\\Valentin Aubry\\01_Data\\Parameters\\Complete_KDE\\TRITC_parameter_space"
fitc_path: "Y:\\Group Members\\Valentin Aubry\\01_Data\\Parameters\\Complete_KDE\\FITC_parameter_space"
```

These paths are used to specify where the parameter iteration space is saved to or loaded from, depending on the setting above.

If you want to generate a new parameter space create a folder (here: Complete_KDE) and enter the folder path to the base path. The program will generate two parameter iteration spaces for the FITC and the TRITC channels with the names: TRITC_parameter_space and FITC_parameter_space.

Note: Newly generated parameter spaces will always have the same name and it will override parameter spaces with these same names.

For loading parameter spaces the folder and the name of the file is required for both the fitc_path and the tritc_path. This means that the files can be renamed after running the program, to avoid being replaced with newly generated ones and to be reloaded later.

n_samples, n_sobol_samples

```
# Number of parameter combinations to test
# Uses farthest-point sampling from valid parameter space
# Higher = more thorough but slower
n_samples: 256

# Internal: Sobol samples for parameter space generation
# Must be power of 2 (e.g., 32768, 65536, 131072)
# Increase if you see "Insufficient valid points" errors
n_sobol_samples: 65536
```

These are some advanced setting that rarely need changing if at all.

n_samples:

- Increase for better coverage of the parameter space (diminishing returns). 256 is already a lot of points.
- Decrease to lower computation time for parameter iteration creation time (less than a minute of time gain) and for adaptive parameter selection (time gain depends on n_calibration_images, but generally also low time gains).

Note: If this is changed, `n_calibration_params` also needs to be changed to match this.

`n_sobol_samples`:

- Increase for better coverage of the parameter space (diminishing returns). These are the points from which the parameter combinations (n-samples) are selected from.
Recommended maximum: 131072 due to diminishing returns with increasing computation time.
- Decrease for lower computation time (less than a minute time gain).

ADAPTIVE PARAMETER SELECTION

`adaptive_parameters`

```
adaptive_parameters:  
# Enable adaptive parameter selection?  
# true: Calibrate → optimize → production (faster)  
# false: Use all n_samples on every image (slower, more thorough)  
enabled: true
```

This setting enables or disables a selection process from all the parameter combinations (n-samples) to get a few that have good performance. This will lower the computation time substantially and is recommended to be set to true for big data sets.

Set this to false, if it is very important to get the absolute best result (not recommended with big data sets). This will lead to a **substantial** increase in computation time.

`n_calibration_images`

```
# Number of images for calibration phase  
# These images test all n_calibration_params to find optimal settings  
n_calibration_images: 2
```

This controls how many pictures are used for the selection of the adaptive parameters. These pictures will be analyzed with all parameter combinations which leads to a higher computation time.

More pictures analyzed this way lead to a more balanced parameter combination selection for the whole data set.

Recommended number of pictures: 3-10 depending on the size of the data set (needs testing)

`n_calibration_params`

```
# Number of parameters to test during calibration  
# Usually same as parameter_space.n_samples  
# If less: uses first N from farthest-point sampling (suboptimal but valid)  
n_calibration_params: 256
```

Only change this setting to match `n_samples`.

`n_production_params`


```
# Number of optimized parameters to use in production
# More parameters = higher diversity coverage
# Algorithm auto-adjusts diversity weighting based on this value
n_production_params: 3
```

Here the amount of the selected parameter combinations can be set. These will be used on the rest of the data set after the calibration images.

Recommended amount: 1-5

- 1: Lowest computation time, least accurate, no detection probability stats for the foci.
- 2-5: Increasing accuracy and computation time.
- 5+: Diminishing returns (needs testing).

calibration_save_path

```
# Where to save calibration results for inspection
calibration_save_path: "Y:\\Group Members\\Valentin Aubry\\01_Data\\Final_Test\\Parameters\\calibration_results.pkl"
```

The path is used to save the full calibration data for the TRITC and the FITC channels with pictures. The names of the specific TRITC and FITC files is automatically generated from the one used in this path. These files will not be used in the program and are only useful for analysis/debugging.

Example

Filename set by the user:

- calibration_results.pkl → base name to generate the other files

Automatically generated names and files:

- calibration_results_TRITC.pkl → TRITC full calibration data
- calibration_results_TRITC_viz.png → Calibration selection picture
- calibration_results_FITC.pkl → FITC full calibration data
- calibration_results_FITC_viz.png → Calibration selection picture

Calibration Control

recalibrate, load_reduced_parameters

```
# Force fresh calibration even if results exist?
# Set true if previous calibration was poor or settings changed
recalibrate: false

# Load pre-saved optimal parameters from previous calibration?
# Completely skips calibration phase
# Requires valid .pkl files at paths below
load_reduced_parameters: true
```

These two settings decide, if the adaptive parameter selection is used to get the optimized parameter combination, or if they are loaded from a previous save file. With this the final parameter combinations can be re-generated from the same parameter space.

To generate a new set of parameter combinations:

- recalibrate: true
- load_reduced_parameters: false

Use this for new data sets.

To load an existing set of parameter combinations:

- recalibrate: false
- load_reduced_parameters: true

Use this if a set of parameter combinations already exists to continue after a break.

Note: Try to avoid setting both options to true or false at the same time.

reduced_tritc_path / reduced_fitc_path

```
# Paths to saved optimal parameters (auto-generated after calibration)
# Contains ONLY the final 1-3 best parameter combinations (not full calibration data)
# Only used when load_reduced_parameters: true
reduced_tritc_path: "Y:\\Group Members\\Valentin Aubry\\01_Data\\Final_Test\\Parameters\\Complete_KDE\\reduced_params_TRITC.pkl"
reduced_fitc_path: "Y:\\Group Members\\Valentin Aubry\\01_Data\\Final_Test\\Parameters\\Complete_KDE\\reduced_params_FITC.pkl"
```

Here the path to the final parameter combinations can be entered to either save a new set or load an existing one.

Example to save a new set of parameter combinations:

```
"Y:\\Group Members\\Example_User\\01_Data\\Final_Test\\Parameters\\Complete_KDE\\new_filename_TRITC.pkl"
```

Example to load an existing set of parameter combinations:

```
"Y:\\Group Members\\Example_User\\01_Data\\Final_Test\\Parameters\\Complete_KDE\\existing_filename_TRITC.pkl"
```

Note: Use the double backslash

Random Seed

randomize_calibration_order

```
# Randomize image processing order?
# true: Shuffle using random_seed (ensures diverse calibration set)
# false: Process alphabetically by filename
randomize_calibration_order: true
```

This setting controls if the pictures in the folder are analyzed in order or randomized.

true: Process images in random order (ensures diverse calibration set)

false: Process alphabetically

random_seed

```
# Fixed seed for reproducibility (null = random each run)
# Keep constant for resume functionality to work correctly
random_seed: 42
```

With this setting a random seed can be entered to get a random, but reproducible result. All the randomizations in the program are connected to this (random picture order, random points in the parameter space etc.)

Set this to 0 to get a random result that is not reproducible (not recommended).

WATERSHED THRESHOLDING

mode

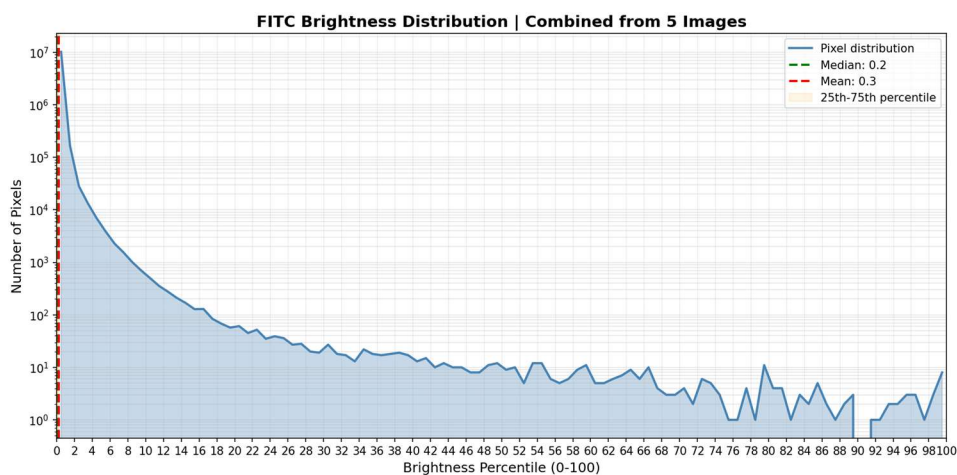
```
watershed:
# Threshold selection mode
# "manual_interactive": Analyze images, show histogram, get user input
# "manual_preset":      Use fixed thresholds below
mode: "manual_interactive"
```

For the watershed a threshold has to be set to limit how big the detected foci will be. With this setting the user can decide between two options how this threshold is set.

manual_interactive:

This means that the brightness of the first few pictures is analyzed and shown to the user. The order of the pictures is the same as how they will be for the main foci detection algorithm. The picture below shows an example of the brightness analysis which distributes all the pixels that belong to a nucleus onto a brightness percentile. Most of these pixels are on the lowest few percentiles and because of this the Y-axis is a logarithmic scale to preserve the detail of the distribution of the brighter pixels, where the border between normal nucleus brightness and foci brightness is.

Example picture for the FITC channel:



With this picture the user can then set a threshold (here for example: 52) in the following input prompt which is then applied to all the pictures in the foci detection workflow.

Use this for:

- New datasets (may need some short test runs on the first few pictures to get an idea of how it looks)

manual_preset:

With this option the user can enter the thresholds for each channel (manual_threshold_tritc, manual_threshold_fitc), which then skips the brightness analysis and directly applies the threshold to the foci detection workflow.

Use this for:

- Continuation of a dataset where the desired thresholds are already known.

n_images_for_threshold

```
# Number of images to analyze in interactive mode
# More images = better representation of dataset variability
n_images_for_threshold: 5
```

This setting decides how many pictures are used for the brightness distribution analysis (manual_interactive mode).

manual_threshold_tritc, manual_threshold_fitc

```
# Fixed thresholds for preset mode (0-100 scale)
# Pixels above threshold = part of focus region
manual_threshold_tritc: 20.0
manual_threshold_fitc: 20.0
```

This setting is only active, if the manual_preset above is used. Here the thresholds for the watershed (foci area) can be entered for both channels.

skip_interactive_config

```
# Skip interactive prompts even in interactive mode?
# Useful for batch processing with predetermined thresholds
skip_interactive_config: false
```

This is a redundant option that has no purpose and was overlooked when reviewing the code. Leave at false.

rescaling_mode

```
# Rescaling mode for watershed segmentation
# "local": Rescale each nucleus independently (0-100)
#           → Better sensitivity for dim nuclei
#           → Foci intensities NOT comparable across nuclei
# "global": Use image-wide intensity scale for all nuclei
#           → Enables cross-nucleus intensity comparisons
#           → Will reduce more dim foci in very dim nuclei to minimal size
rescaling_mode: "global"
```

The foci detection workflow uses brightness percentiles to set thresholds for various things (foci detection and area detection). For this the pixel brightness needs to be converted from the numerical value it has in the picture to the percentile. This setting decides how this conversion is done.

Local:

This means that all the nuclei are individually rescaled from 0-100. This means that the foci with brightness 80 might be a 50 in the neighboring nucleus.

Advantages:

- Detects more foci in dim nuclei (better sensitivity)

Disadvantage:

- Foci intensities and sizes are not comparable between different nuclei

Global:

With this option all the nuclei are using the same scale, this means that most nuclei will not contain the full range from 0-100.

Advantages:

- All the foci are comparable with each other in intensity and size between different nuclei.

Disadvantages:

- Less foci detection in dim nuclei.

VISUALIZATION

mode

```
visualization:
# Visualization mode
# "all":      Every image (slow, large storage)
# "none":     No visualizations (fastest)
# "specific": Only images in lists below
mode: "all"
```

The program is able to save pictures of the detected foci locations and their area for both channels (FITC, TRITC) individually. Here the user can decide the mode on how many pictures should be saved. Saving a picture greatly increases the computing time and is not advised for a full data set.

all (slowest):

- For small data sets where you need all the pictures.
- When max_images (Processing Control) is set to a low testing size.

none (fastest):

- For full data sets where all the settings are tested and correct.

specific (balanced):

- To get result pictures from a select few images when running a bigger data set.

specific_images, specific_basenames

```
# Specific images by well/position (format: "WELL_POSITION")
# Example: ["00029_00035", "00044_00021"]
# Use [[]] for empty list
specific_images:
  - []

# Specific images by filename prefix
# Example: ["ATR1_24h--W00032--P00015--Z00000--T00000--"]
# Use [[]] for empty list
specific_basenames:
  - []
```

Here the specific images can be selected, where a picture should be saved. Only works if the mode “specific” has been selected above. There are two ways to specify the image, either by entering the well number and the position number for specific_images, or by entering the base name for specific_basenames. Pictures will be saved for both lists so they only need to be listed in one of the lists and not both.

For this to work the naming convention of the images should be like this:

Condition_Time--Well number--Position number--...--Channel name SEM.tif

Examples :

ATR3_8h--W00053--P00001--Z00000--T00000--DAPI SEM.tif

UT_8h--W00017--P00034--Z00000--T00000--FITC SEM.tif

ATR4_8h--W00065--P00009--Z00000--T00000--Cy5 SEM.tif

Example lists

specific_images:

- "00053_00001"
- "00017_00034"
- "00065_00009"

specific_basenames:

- "ATR3_8h--W00053--P00001--Z00000--T00000--"
- "UT_8h--W00017--P00034--Z00000--T00000--"
- "ATR4_8h--W00065--P00009--Z00000--T00000--"

Use – [] for empty lists (like in the picture of the code).

watershed_min_detection_prob

```
# Minimum detection confidence for watershed visualization (0-100%)
# Foci detected by fewer than this % of parameters are excluded
# 0 = show all detected foci
watershed_min_detection_prob: 0.0
```

Every focus has a detection probability that is calculated based on how often it was detected in all the tested parameter iterations ($\frac{\text{all tested parameter combinations}}{\text{instances where the focus was detected}}$). Here a threshold can be set to exclude all the foci that were detected less often. For this to be useful the number of tested parameter combinations need to be greater than 1.

Use this for:

- Generating a picture for comparisons/presentations

Note: The detection probability is recorded and saved in the data anyways so this function is only really useful for generating specific pictures.

TEXTURE FILTERING

enabled

```
texture_filtering:  
  # Enable texture-based filtering?  
  enabled: true
```

Texture filtering uses the coefficient of variation (CV) to determine how uniform the texture of the nuclei is. With this enabled nuclei below a certain threshold (low CV = more uniform) can have a more strict filter applied to them. The foci will need to be brighter than the background by a bigger margin. The advantage is that in uniformly bright nuclei less false detections occur.

Note: The CV value is saved even if the texture_filtering is disabled, so it is possible to exclude foci with a low CV value manually after the fact.

min_cv_threshold

```
# Minimum CV (coefficient of variation) for "textured" nucleus  
# CV = std / mean (range 0.0 - 1.0)  
# Nuclei below threshold get stricter filtering  
min_cv_threshold: 0.20
```

This threshold decides at which CV value the stricter foci detection filter is applied.

Note: This feature has not been tested at all. Value ranges would need to be tested from scratch.

uniform_contrast_multiplier

```
# Contrast multiplier for uniform nuclei (>= 1.0)  
# Example: contrast_threshold × multiplier = adjusted threshold  
# Higher = more aggressive filtering of uniform nuclei  
uniform_contrast_multiplier: 1.0
```

This is the additional multiplier that is added to the contrast_threshold (how much brighter than the background the foci has to be). This is only applied to the foci that are in regions that have a CV below the above set threshold.

Note: This feature has not been tested at all. Value ranges would need to be tested from scratch.

PROCESSING CONTROL

max_workers

```
processing:  
  # Number of parallel workers (null = auto: CPU_cores - 1, max 8)  
  # Reduce if memory errors occur  
  max_workers: null
```

This setting decides how many parallel workers the program will use within a range of 1 to 8 workers. More workers will lead to a faster completion time.

null(auto) behavior:

4 core CPU: 3 workers

8 core CPU: 7 workers

16/32 core CPU: 8 workers

Keep this at null (auto) unless an out-of-memory error pops up, then try to reduce it.

max_images

```
# Maximum images to process (null = all images)
# Useful for testing or incremental processing
max_images: null
```

With this setting the amount of images processed can be set. This means that the first X images will be processed. This can be useful to test a setting or data set by just running the first few pictures. By setting it to null the full data set will be analyzed.

Note: The order of the first few pictures will be randomized if the `randomize_calibration_order` setting is set to true. This means that the first few pictures in that random order will be processed.

resume_from_image

```
# Resume from image N (0 = start from beginning)
# CRITICAL: Keep random_seed constant across resume runs
#
# CSV behavior:
#   resume_from_image > 0: Data appended to existing CSVs
#   resume_from_image = 0: Prompts if CSVs exist (overwrite/rename/abort)
resume_from_image: 0
```

With this setting the starting point of the image analysis can be set. The main use case will be to continue analyzing a data set that was started in an earlier run. For this the data will automatically be appended to the already existing .csv with the same name in the folder. If it is set to 0 and a .csv with the same name already exists, a prompt to change the name of the new file or to overwrite the old one will appear at the end of the run.

Note: If the order of the pictures is randomized, the run can be continued if the random seed is kept the same between the two runs.

After all this setup the program is ready to run. The following steps take place after the program is started (some steps need to be enabled in the settings, so if you expect a new parameter space generation or manual setting of foci brightness thresholds check the settings).

Parameter space generation

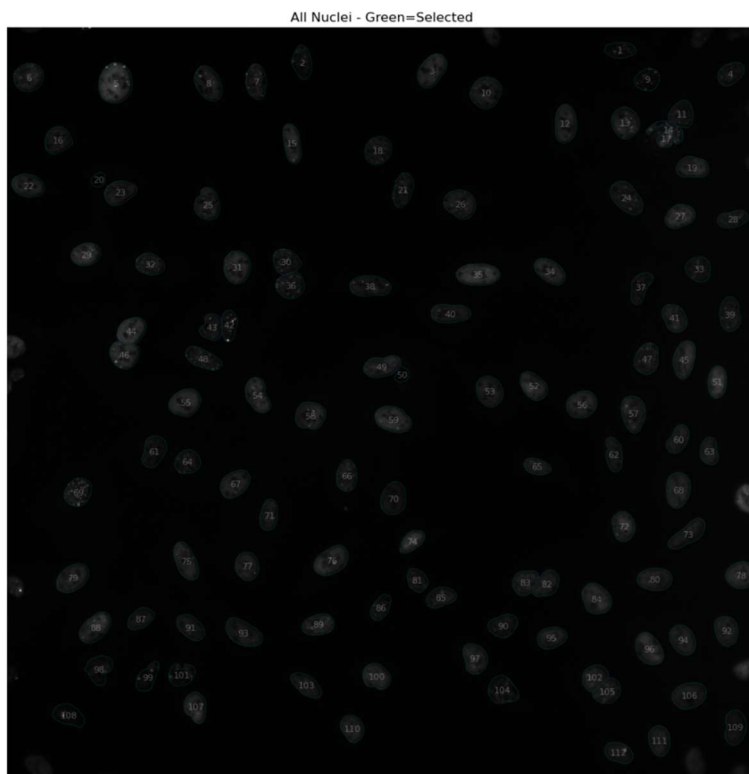
For this part of the program to take place the option `generate_new` in the settings needs to be set to true. If you want to load an existing parameter space and this part of the program starts up, interrupt the program, change the settings and then start the program again.

In this part of the program a new parameter space for the TRITC and the FITC channels can be created for a specific data set. For this the user can select up to 10 (2-3 recommended) nuclei and tell the program how many foci should be detected. Then the program will iterate through a range of parameter combinations and create a parameter combination space within which all the selected nuclei have their foci numbers correctly detected. In a later step the program can pick the best performing few parameter combinations from that space to use for the whole data set.

A. Nuclei overview

In this first step the first picture from the data set will be loaded and shown (order can be randomized in the settings). All the (detected) nuclei have a number with which they can be selected to add a foci number ground truth to them. If there are not enough good examples, it is possible to switch to another picture by following the instructions given.

Example Overview:



Note: It can be helpful (to save time) to look at the .tif file itself (outside of the program) to find nuclei with clear foci counts, as it can be a bit difficult to see in the overview picture provided by the program. The overview picture is mainly there to number each nucleus for selection.

Important: Make sure that all the selected nuclei are from the same picture as the program does not have the functionality to analyze more than one picture at once. It does not matter which picture is chosen in the end but all the selected nuclei need to be from the same one.

B. Selecting Nuclei

Selecting a nucleus will show three close up pictures of the nucleus: The first one without any modifications, the second one with the difference of Gaussian filter (removes background noise) applied and the third (red) one with an increased contrast.

Then the user can enter the number of expected foci, this can be done as a singular number or by entering an acceptable range of foci (example: min-max).

With the command “abort” the user can exit out of the selection without entering a foci count to select a different nucleus.

With the command “abort+” the user can also exit out of the selection without entering a foci count, but additionally the overview picture will be printed again. Use this if the scrolling distance to the overview is too long.

Example nucleus selection:

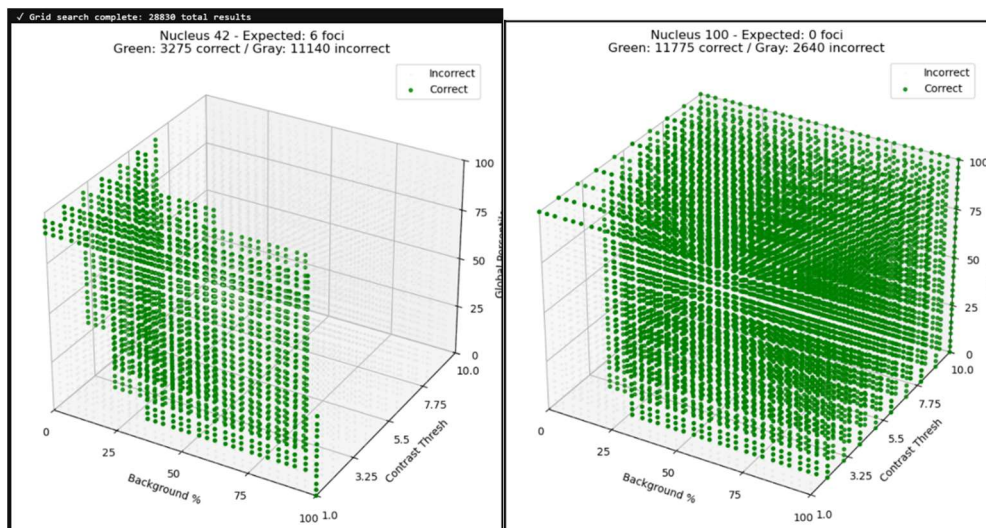


C. Verifying Results

After selecting up to 10 (2-3 recommended) nuclei and entering their foci count the command “done” starts the automatic parameter space calculation.

At first a parameter sweep takes place selecting all the parameter combinations that are resulting in the specified foci counts being detected. All the parameter combinations that match the foci counts are marked as green dots.

Example for 2 nuclei:



Analyzing the example results

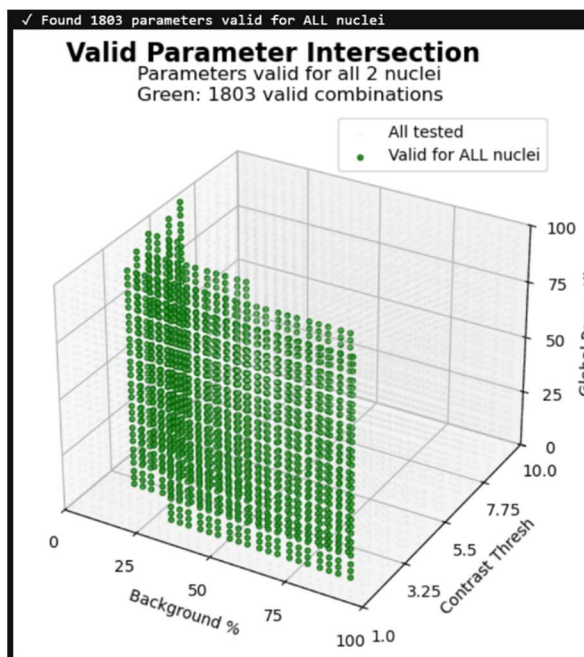
Nucleus 42: Good number of green points that are connected. Could be problematic with a high number of selected nuclei.

Nucleus 100: Foci count was not restrictive enough as 0 foci was the expected result, but in combination with nucleus 42 not a problem.

Note: If there are no or almost no green points for one nucleus, restart the program and either choose a different nucleus or enter a less restrictive (but still valid) number of expected nuclei.

In the next step the program compares all the correct (green) parameter combinations and combines them, keeping only those that were correct in all the selected nuclei.

Example:

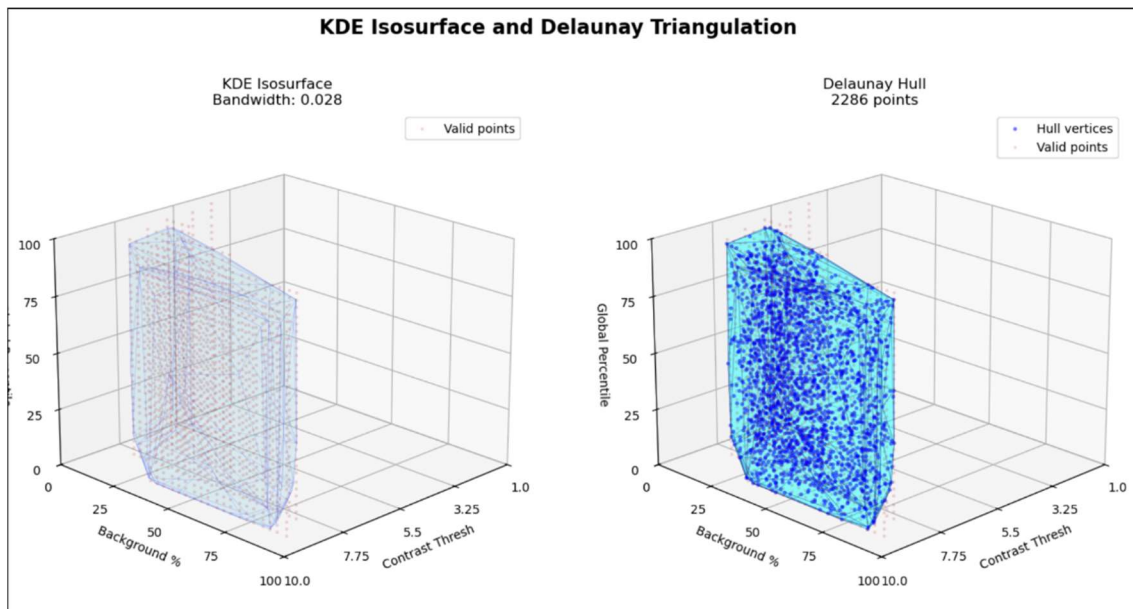


Analyzing the example result:

- Connected body of points with a clear parameter space restriction. Very promising result.

Note: Selecting more nuclei can lead to a smaller valid parameter intersection which could impair the functionality of the program. If the valid parameter intersection is much smaller than in the example or in disconnected chunks, try lowering the amount of selected nuclei.

Finally the program will create the parameter space by removing the outer 15% of all the remaining parameter combinations to eliminate outliers.



Analyzing the example results:

KDE Isosurface: Parameter space (blue) matches the valid parameter intersection (red points).

Delaunay Hull: Decently sized parameter space without any disconnected parts.

After this the parameter space can be saved, or the program can be restarted to try again.

Foci area detection threshold

This option can be enabled by setting mode: "manual_interactive" in the watershed section.

Here the user can set a threshold for the area detection of the foci for each the FITC and the TRITC channels.

The first few pictures (adjustable under `n_images_for_threshold`) are analyzed and the pixels that are part of a nucleus (background is excluded) are shown as a distribution according to their brightness percentile from 0-100. The user can then set a threshold and all the pixels that are above that threshold and connected (via other pixels above the threshold) to a detected focus center are counted as the area of that focus.

This generally means that:

High threshold:

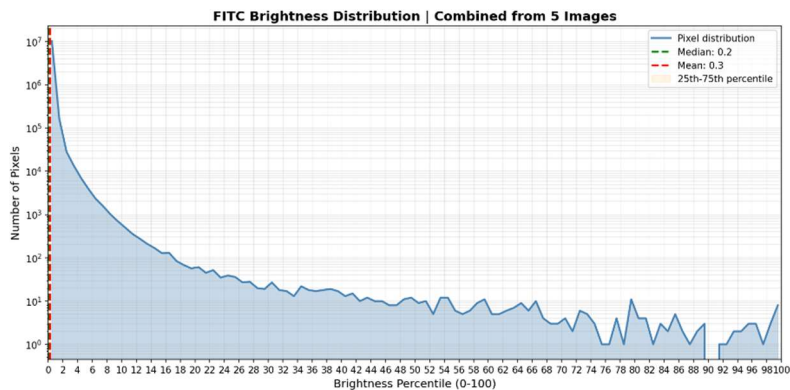
- Small foci areas
- All the foci with only pixels below the threshold around them will be detected with only 1 pixel area size → Loss of detail on dim foci

Low threshold:

- Big foci areas
- Foci with a lot of bright pixels around them can “spill over” and have big misshapen areas.

The goal here is to find a balanced approach. A good starting point is around the 10^1 Number of pixels on the Y-axis to do a test run on a few pictures, while saving the output pictures, for a quick visual check.

Example picture for the FITC channel:



Example analysis:

With this brightness distribution a threshold between 34 – 48 would be a good starting point.

Picture Analysis

After this the program will automatically analyze all the pictures as specified in the settings. A progress bar will be displayed, to help with estimating the end time.

Important: Check for any input prompts at the end of the program. If the excel-files with the same name as in the settings already exist in the target folder the name can be changed before saving the results.